

Evolution status after Kona 2019

Abstract

This paper is a collection of items Evolution has worked on up to and in the Kona meeting, their status, and plans for the future.

Concepts

No new activity, EWG is in bug-fix mode for Concepts.

Modules

The merged Modules proposal, [P1103R3](#), has been merged. There is SG15 (Tooling) agreement to create a TR for tool and build system solutions related to Modules. We also approved D1498R0, Constrained Internal Linkage for Modules, for C++20. Other than that, EWG is in bug-fix mode for Modules.

Coroutines

The Coroutines TS has been merged. EWG is in bug-fix mode for Coroutines.

Aggregate initialization

The proposal to allow paren-initializing aggregates, [P0960](#), has been merged.

Contracts

P1290R1, Avoiding undefined behavior in contracts, and P1429R0, Contracts That Work were discussed, but failed to gain consensus. In addition, P1320R1, Allowing contract predicates on non-first declarations was discussed, but failed to gain consensus. EWG is in bug-fix mode for Contracts, but we expect various proposals to arrive for discussion in the Cologne meeting.

Some constexpr extensions

We approved P1306R1, Expansion statements, and P0784R5, More constexpr containers, the latter without non-transient allocation, for C++20. We also approved P1331R0, Permitting trivial default initialization in constexpr contexts, for C++20.

Template argument deduction

No current activity, EWG is in bug-fix mode for CTAD.

All Your Spaceships Are Belong to Us

We discussed P1186R1, When do you actually use `<=>`?, and approved it for C++20 with a tweak to synthesize strong and weak ordering only from `==` and `<`. We also discussed P1186R1, When do you actually use `<=>`?, and approved it for C++20. This means that defaulting `<=>` unconditionally declares a defaulted `==`.

Structured bindings

We discussed P1481R0, constexpr structured bindings, and it failed to gain consensus for C++20. There's encouragement to do further work on the problem.

Guaranteed constant initialization for non-constexpr variables

We approved P1143R1, Adding the `constexpr` keyword, for C++20.

Deprecations

We approved D1152R2, Deprecating `volatile`, for C++20.

Using enum

We approved P1099R3, Using Enum, for C++20.

Implicit object creation

We approved D0593R4, Implicit creation of objects for low-level object manipulation, for C++20.

Tweaks to move semantics

We approved P1155R2, More implicit moves, for C++20.

Near-future EWG plans

The high-order bit of our next steps are any necessary design clarifications and bug fixes, to happen in Cologne. Beyond that, possibly already starting in Cologne, Evolution will switch into C++23 mode while also doing ballot resolution for C++20 as necessary.